

극딜 용사 — AI 활용 기술 문서

NHN NAN 2026 게임잼 제출용 · AI 도구·프롬프트·활용 내역 정리 작성일 2026-08-10 · 대상
빌드: `godot-game` (Godot 4.7.1.stable)

1. 개요

1.1 프로젝트 기본 정보

항목	내용
게임 제목	극딜 용사
장르	실시간 액션 로그라이크 (5칸 딜싸이클 전투 + 무한 필드 탐험)
엔진 · 언어	Godot 4.7.1.stable · GDScript
기준 해상도	1280×720 · GL Compatibility
개발 기간	2026-08-03 – 2026-08-10 (8일)
팀 구성	인간 디렉터 1명 + AI 에이전트 (주 개발 Claude Code · 교차 검증 OpenAI Codex CLI)
저장소	<code>https://github.com/ALXxxXNDR/extreme-hero</code> — 전체 소스와 커밋 기록을 공개한다
코드 규모	GDScript 46개 파일 · 약 48,900줄
문서 규모	인수인계·설계 문서 약 21,100줄 (핸드오프 문서 36개 포함)
콘텐츠	스킬 카드 28종 · 장비 57종 · 각인 15종 · 몬스터 10종 · 보스 3종 + 마왕 · 필드 사건 8종 · 소비 아이템 8종 · 5스테이지
자동 검사	컴파일 1 + 기능 검사 16종 · 시각 캡처 15종 73컷 · 밸런스 프로브 5종

1.2 한 문장 요약

게임 코드는 AI가 100% 작성했고, 인간은 콘셉트·제약·판정 기준을 정한 뒤 실제 플레이 화면으로 최종 승인했다.

인간 디렉터는 GDScript를 한 줄도 타이핑하지 않았다. 대신 다음 네 가지를 했다.

1. 무엇을 만들 것인가를 정한다 (콘셉트·차별화 축·게임의 정체성)
2. 무엇을 하면 안 되는가를 정한다 (저작권 안전선, 어휘 금지어, 화면 점유 상한)
3. 실제 창을 띄워 플레이하고 난이도·가독성·재미를 판단한다
4. AI가 제출한 검증 결과를 승인하거나 반려한다

1.3 사용한 AI 도구

도구	용도	산출물이 최종 빌드에 들어가는가
Claude Code (Anthropic · 에이전트형 코딩 CLI)	주 개발 에이전트 — 설계·구현·테스트·디버깅 전 영역. 추론 모델은 Claude Opus 계열	예 — 게임 코드 전량
OpenAI Codex CLI	교차 검증·보조 구현 — 주 에이전트 산출물의 코드 리뷰와 대안 진단	일부 — 리뷰 지적을 반영한 수정과 보조 구현분
OpenAI 이미지 생성	비주얼 방향 참고용 콘셉트 시트 1장	아니오 — 방향 참고용, 런타임 미사용
Google Gemini	BGM 원곡 생성 — 배경 음악의 원곡을 만든다	예 — 배경 음악
Suno	BGM 리믹스·편곡 — Gemini 원곡을 게임에 맞게 다듬는다	예 — 배경 음악
Godot 4.7.1 headless	AI가 작성한 자동 검증 테스트의 실행기 (<code>--headless</code> 모드)	검증 도구 (빌드 산출물 아님)

도구를 둘로 나눈 이유는 단순하다. 코드를 쓴 모델과 그 코드를 의심하는 모델을 같은 것으로 두면, 같은 가정에서 나온 실수는 끝까지 발견되지 않는다. 그래서 작성은 **Claude Code**가 전담하고, 위험한 구간(밸런스 상수·전투 판정·저장 스키마)은 **Codex CLI**에 다시 읽혀 두 번째 의견을 받는 방식을 썼다. Codex는 보조 역할이며, 게임의 설계·구현·검증 본류는 Claude Code가 담당했다.

- 런타임 그래픽은 **Godot Draw API**로 그린 오리지널 도형 + **CC0** 픽셀 에셋이다.
- 효과음은 `AudioStreamWAV` 샘플을 런타임에서 합성한다.
- 배경 음악은 **Gemini**로 원곡을 생성하고 **Suno**로 리믹스했다.

- 외부 상용 게임 에셋은 0건이다.
-

2. AI와 인간의 역할 분담

아래 표가 이 프로젝트의 협업 계약이다. **“AI가 만들고 인간이 고른다”**가 아니라, **“인간이 경계를 긋고 AI가 그 안을 채운 뒤 스스로 검증하고, 인간이 화면으로 최종 판정한다”**에 가깝다.

영역	AI가 한 일	인간 디렉터가 한 일
컨셉트	장르 조합·선택 시스템 후보 다수 제안	***고르지 않은 것이 전부 마왕에게 간다***는 인과 구조 확정
장르	유사 장르 사례 조사와 차별화 후보 제시	탐다운 액션 로그라이크로 확정, 범서류와의 차이를 직접 규정
엔진	Phaser·Unity·Godot 세 엔진의 장단점 표 작성	Godot 최종 선택
전투 코어	5칸 딜사이클 · 바늘 런타임 · 각인 15종 엔진 · 상태이상 5종과 반응 11종 구현	동시 발동 폐기 지시, ”한 칸만 나간다” 규칙 확정, 과열 시스템 폐기 지시
월드 생성	무한 좌표 청크(Simple Tiled WFC) · 호수 · 바위 · 다리 · 랜드마크 3종	”바위와 물은 플레이어도 몬스터도 못 지난다” 등 통행 규칙 확정
필드 생태	습성 5종(무리·겁쟁이·텃세·매복·사냥) · 지형 가중 스폰 · 돌 우회 AI	”1·2스테이지 낮에는 아무도 먼저 덤비지 않는다” 규칙 지시
성장 시스템	카드 2택1 · 취소 선택지 · 각인 드래프트 · 보스 트로피 5종 · 마왕 자동 합성	버린 것의 귀속 범위(스킬→장비→각인 조각)를 단계적으로 확장 지시
시각 · UI	UI 킷 10장, 픽셀 아이콘 생성 코드, 프레임리스 HUD, 호버 툴팁 컴포넌트	”블록을 다 지워라”·”중학생이 3초 안에 알아보게” 같은 판정 기준 제시
사운드	BGM 원곡 생성(Gemini)과 리믹스·편곡(Suno) · <code>AudioStreamWAV</code> 런타임 합성 효과음 구현	곡 분위기 지시와 채택 판정, 톤·볼륨을 실제 플레이에서 판단
테스트	headless 기능 검사 16종 · 캡처 15종 · 밸런스 프로브 5종을 AI가 직접 설계·구현	통과 기준 승인, 캡처 육안 검수, ”이건 통과했는데 재미가 없다” 판정
밸런싱	<code>balance_probe.gd</code> 로 DPS·TTK·골드 수치·체류 압박 정량 측정 후 상수 조정	목표 창(관문 30–60초 · 마왕 60–120초 · 15–20일 클리어) 확정
문서화	핸드오프 문서 36개 · 설계 문서 · 변경 로그 자동 작성	문서 체계 자체를 요구 (AI 세션이 끊겨도 이어지도록)

여기서 “AI”는 대부분 주 개발 에이전트인 **Claude Code**를 가리킨다. **OpenAI Codex CLI**의 지적은 그대로 반영하지 않고, 인간 디렉터가 채택 여부를 판정한 뒤 주 에이전트에게 되돌려 반영·재검증하게 했다.

3. 개발 워크플로우 — 이 프로젝트의 방법론

AI를 “코드 생성기”로 쓰면 8일짜리 게임잼에서 이 볼륨이 나오지 않는다. 다음 네 가지 장치가 실제로 속도를 만들었다.

3.1 세션 단위 개발 + 핸드오프 문서 체계

AI 에이전트 세션은 컨텍스트가 유한하고, 세션이 끝나면 기억이 사라진다. 그래서 작업을 “웨이브(wave)” 단위로 쪼개고, 웨이브가 끝날 때마다 다음 AI 세션이 읽을 인수인계 문서를 AI가 직접 작성하게 했다.

AGENTS.md ← 단일 인수인계 원본(Single Source of Truth). 모든 세션이 먼저 읽는다
 ↳ docs/handoff-*.md ← 웨이브별 상세 인수인계 36개

시리즈	문서	다룬 범위
W 계열 10개 (w3 없음)	handoff-w*.md	v2 — 딜사이클 레일 전투의 초기 구축
V1-V10	handoff-v*.md	v3 — 5스테이지 개편 · 상태이상 · 보스 3종 · 트로피 · 저장 스키마 v3
U1-U3	handoff-u*.md	UI 킷 전면 재스킨 + 스포트라이트 온보딩 길잡이
X1-X4	handoff-x*.md	1차 플레이테스트 피드백 — 레벨업 모달 · 편집 화면 · HUD · 온보딩
Y 계열 9개 (YZ 없음)	handoff-y*.md	2차 플레이테스트 피드백 25건 — 과열 폐기 · 각인 15종 · 필드 생태 · 타격감 · 밸런스 실측

문서 규격은 AGENTS.md §14에 못 박아 두어 모든 웨이브가 같은 형식을 쓴다.

- 사용자 요청: - 상태: `completed` | `in_progress` | `blocked`
- 결정한 내용: - 변경 파일: ``path/to/file``: 무엇을 왜 바꿨는지
- 사용자에게 보이는 변화: - 데이터/저장 호환성:
- 실행한 검증: ``정확한 명령`` → `PASS/FAIL/미실행` (출력 원문 첨부)
- 밝은 함정: - 이 웨이브가 하지 않은 것:
- 다음 AI가 시작할 정확한 지점:

특히 「**밝은 함정**」 항목이 핵심이다. `handoff-y5.md` §5-5는 “`enemy.gd`에 남은 디버그 `print`가 `=false` 문자열을 찍으면 `run_all.sh`가 통째로 FAIL이 된다”는 사실을 남겼고, 이 한 줄 덕분에 이후 세션이 같은 원인으로 헤매지 않았다.

나쁜 기록의 예시까지 문서에 박아 두었다 — “UI 수정함. 테스트 대충 됨. 다음에 계속.” 이런 문장이 나오면 인수인계 실패로 본다.

3.2 자동 검증 우선 — AI가 만든 코드를 AI가 만든 테스트로 검증

인간이 매번 게임을 켜서 확인하는 것은 8일 일정에서 불가능하다. 그래서 **AI가 기능을 만들 때 그 기능을 무는 headless 검사를 같이 만들도록** 했고, 밸런스는 별도 프로브가 숫자로 회귀를 막게 했다.

```
bash godot-game/scripts/test/run_all.sh      # 컴파일 1 + 기능 검사 16종 (약 135초)
godot --headless --path godot-game -s res:///scripts/test/balance_probe.gd
# 약 90초
```

- 종료 코드 **0이면 전부 통과**. 다음 AI 세션은 이 숫자만 보면 된다. FAIL로 집계하는 조건 네 가지는 §5.2 표에 정리했다.
- 검사 항목: 월드 · 필드 생태 · 5칸 텍 · 성/트로피 · 균열 · 스트레스 · 스모크 · 전투 · 스테이지 클럭 · 상태이상 · 사이클 런타임 · 각인 드래프트 · 보스 3종과 마왕전 · 이어하기 · 길잡이 · 필드 사건
- `balance_probe`는 12개 절로 나뉘어 화력·몬스터 HP 복리 지수·체류 곡선·물량 상한·경제·보스 TTK·마왕 TTK를 재고 마지막 줄에 `**pass=1**`을 찍는다. `AGENTS.md`는 `***밸런스 숫자를 바꾸기 전에 먼저 돌린다***`를 규칙으로 못 박아 두었다.

3.3 인간은 “실제 창”으로 판단한다

headless 검사는 `***무너지지 않았다***`를 증명하지만 `***재미있다 · 읽힌다 · 어렵다***`는 증명하지 못한다. 그 축은 전부 인간이 맡았다.

- AI가 `--captures` 모드로 실제 창을 띄워 **15종 73컷**의 PNG를 저장하고, 인간이 그 컷을 열어 보고 판정한다. 판정 기준도 문서에 문장으로 남긴다.
 - 편집 화면 **”중학생 3초 테스트”** — 컷 1장만 보고 ①5칸이 순서대로 돈다 ②카드를 끌어 넣는다 ③각인이 칸에 붙어 있다 를 알아차리는가
 - HUD 필수 검수 컷은 `hud-x3-stage5-night.png` — 가장 어두운 화면에서 체력 바·바늘·화살표·숫자가 전부 읽히면 나머지는 자동이다
 - 온보딩 **”한 페이지를 3초 안에 훑을 수 있는가”**
- 인간이 직접 플레이한 뒤 피드백을 문서로 넘긴다. 2차 라운드는 **피드백 25건**을 `docs/FEEDBACK_Y.md`로 정리했고, 그 문서의 §11이 **”25건 전부 배정됐다. 누락 0.”**으로 추적을 마무리했다.

3.4 정성적 판단도 검사 계약으로 바꿨다

가장 특이한 장치는 **”읽기 힘들다”**는 감각을 숫자 계약으로 번역한 것이다.

감각적 요구	코드가 매 실행 찍는 지표	계약
”HUD가 필드를 가린다”	<code>hud_block_pct</code> / <code>hud_ink_pct</code>	블록 면적 상한 3.35% 계약 · 실측 3.19% (잉크 31.74% → 6.39%, 약 5배 감소)
”편집 화면에 글이 너무 많다”	<code>edit_labels</code> / <code>edit_prose</code>	상시 문장 라벨 0건 · 글자 라벨 32개 이하
”온보딩이 길다”	<code>onb_peak</code> / <code>onb_longest</code>	페이지 ≤360자 · 한 줄 ≤32자 · 규칙 ≤2줄

HUD 수치는 상시 9-slice 판 5장을 **0장**으로 줄이고 딜사이클 스트립을 1048×156에서 **358×74**로 축소한 결과다. 한 번 숫자가 되면 **회귀가 자동으로 잡힌다**. 다음 세션이 설명을 한 줄 늘리는 순간 검사가 빨간불을 켜다. 이것이 “AI가 만든 UI가 세션마다 조금씩 다시 복잡해지는” 문제의 구조적 해법이었다.

4. 대표 프롬프트

§3이 **”어떤 구조로 일했는가”**라면, 이 절은 **”그 구조 안에서 무엇을 어떤 문장으로 시켰는가”**다. 프롬프트는 “요청문”이 아니라 **작업 지시서**에 가까웠고, 8일 안에 이 볼륨이 나온 이유의 절반은 프롬프트마다 아래 네 가지를 **처음부터 같이 준** 데 있다.

구성	무엇을 적는가	없으면 생기는 일
① 목표	이 작업이 무엇을 만드는가	AI가 인접한 더 쉬운 문제를 대신 푼다
② 제약·금지선	바꾸면 안 되는 것, 되돌아가면 안 되는 구조	매 세션 조금씩 원래대로 돌아온다
③ 상한·가드	규칙의 수치 상한과 밸런스 안전장치	무한 루프와 몰빵 조합이 나중에 터진다
④ 완료 판정	무엇이 초록불이어야 “끝”인가	”된 것 같다”를 사람이 매번 확인해야 한다

같은 형식의 짧은 교정도 같은 방식으로 돌아갔다. 아래 둘은 인간의 한 문장이 시스템 하나를 새로 만든 사례다.

디렉팅	AI 구현	검증
낮 필드는 몬스터가 상주하고 플레이어가 찾아가는 구조여야 한다. 1·2스테이지 낮에는 먼저 덤비지 않는다	습성 5종(무리·겁쟁이·텃세·매복·사냥)을 기존 행동 AI 위에 직교 축으로 얹고, 지형이 종을 정하는 가중 스폰과 돌 우회 경로를 구현	<code>--field-test</code> 신설 → <code>habit_wired</code> · <code>habit_day</code> · <code>day_aggro_zero</code> · <code>terrain_spawn</code> · <code>herd_spawn</code> · <code>rock_detour</code> 전부 true
무조작 검사가 낮 12초 만에 죽는 난이도는 반려	원인을 낮 추적형 몬스터 비율 25%로 특정하고, 낮 생성 확률을 도주 36% / 반격 32% / 무리 23% / 추적 9%로 재배분	<code>--combat-test</code> → 12초 실제 물리·AI 실행 후에도 <code>state=playing</code> . 이 축은 이후 <code>balance_probe</code> 의 체류 압박 곡선으로 정식화

아래 네 개 프롬프트는 실제 개발에서 내린 디렉팅을 **제출용으로 정제한 것**이다. 근거가 된 원문 디렉팅과 그 구현 대조표는 `docs/handoff-*.md` 36개 문서에 “사용자 원문 → 구현” 표로 그대로 남아 있다.

4.1 프롬프트 1 — 게임의 정체성을 못 박다

상황: 뱀서라이크를 피하려던 AI가 게임을 **벽돌깨기**로 바꿔 놓은 시점.

너는 이 게임의 구현 담당이고, 장르와 차별화 축은 내가 정한다.

[목표] 흔한 뱀서라이크와 다른 게임을 만든다. 단, 차별화 수단은 "장르 변경"이 아니다.

[확정 규칙 — 이걸 바꾸지 마라]

1. 장르는 탑다운 실시간 액션 로그라이크로 고정한다.
벽돌깨기·퍼즐·리듬 같은 다른 장르로 갈아타지 마라.
2. 차별화 축은 단 하나다 — "플레이어가 고르지 않은 선택은 전부 마왕이 가져간다."
레벨업에서 버린 스킬, 교체하며 버린 장비가 그대로 최종 보스의 힘이 된다.
3. 그래서 플레이어의 성장과 마왕의 강함은 같은 자원에서 나온다.
이 인과가 끊기는 설계는 전부 반려한다.

[해야 할 일] 위 규칙 셋을 만족하는 최소 실행 버전을 Godot으로 다시 짜라.

[완료 판정] 한 판을 자동으로 끝까지 돌리는 검사를 함께 만들고,
"버린 개수"와 "마왕이 받은 개수"가 일치하는지를 그 검사가 출력으로 단언해라.

AI가 수행한 것 · 검증: Godot 기반 탑다운 액션으로 전면 재구현하고, 귀속 범위를 버린 스킬 → 버린 장비 → 버린 각인 조각까지 세 번에 걸쳐 확장했다. 한 판을 자동으로 도는 `--smoke-test` → `state=won skills=4 rejections=4 boss_items=1`. 이후의 모든 시스템이 이 규칙 위에 얹혔다.

4.2 프롬프트 2 — 중심 규칙 교체: 동시 발동 폐기 → 5칸 딜사이클 레일

상황: 스킬이 쿨타임마다 각자 터지니 플레이어가 자기가 뭘 하고 있는지 볼 수 없었다.

지금 전투는 "스킬이 쿨타임마다 각자 터지는" 구조다. 이 구조를 폐기하고 아래로 바꿔라.

[바꿀 것] 스킬을 칸에 꽂는 레일 하나로 만든다.

- 바늘이 1번 칸부터 오른쪽으로 한 칸씩 이동하며 그 칸의 카드를 실행한다.
- 마지막 칸을 넘어가면 한 바퀴가 끝나고, RELOAD 시간을 지불한 뒤 다시 1번 칸으로 간다.
- 빈 칸은 기본 공격이다. 공격이 나가는 도중에는 방향만 회전하고 타이머는 유지된다.

[유지할 것] 기존 카드·아이템 데이터와 저장 파일은 최대한 살려라. 새로 만들 이유가 없다.

[불변식] "지금 무엇이 나가고 있는가"가 화면에서 항상 한 곳으로 읽혀야 한다.

여러 개가 동시에 터지는 구조로는 어떤 이유로도 되돌아가지 마라.

[완료 판정]

- 바늘 런타임을 실시간으로 도는 headless 검사를 만들고, 한 바퀴의 실행 순서·지속시간·RELOAD 길이를 출력으로 남겨라.
- 지속형 근접 공격이 같은 적을 한 바퀴에 두 번 때리지 않는지 함께 단언해라.
- 실제 1280×720 창은 내가 직접 띄워 최종 판정한다.

AI가 수행한 것 · 검증: 카드 데이터 · 레일 런타임 · 지속시간과 RELOAD 분리 · 지속형 근접 히트 윈도우와 타격 ID 기록(중복 타격 방지)을 구현했고, 이 구조가 이후 5칸 고정 + 각인 15종 + 속성 콤보로 성숙했다. `--cycle-test` (바늘 런타임 · 실시간 12초) · `--v4-test` (5칸 덱) 통과 + 실제 창 확인. 게임 제목이 `THE UNCHOSEN` → 딜싸이클 용사 로 바뀐 지점이 여기다.

4.3 프롬프트 3 — 규칙 상한과 밸런스 가드를 미리 박다 (과열 폐기)

상황: 각인이 바늘을 되돌려 한 바퀴가 무한히 길어지는 것을 **확률 감쇠(과열)** 로 막고 있었고, 플레이어는 왜 각인이 안 터지는지 이해할 수 없었다.

[문제] 지금 "과열"은 확률로 각인을 죽인다. 플레이어가 원인을 볼 수 없으니
이건 규칙이 아니라 노이즈다.

[지시] 과열을 상수·함수·결과 키·화면 문구까지 전부 삭제하고, 아래 한 줄로 대체해라.
▶ "한 칸은 한 바퀴에 두 번까지만 밟는다."

[제약]

- 각인은 24종에서 15종으로 줄인다(칸 각인 10 + 레일 각인 5). 겹치는 개념은 합쳐라.
- 같은 각인을 몰빵했을 때 이득이 선형으로 계속 커지면 안 된다. 사본에는 감쇠를 넣어라.
- RELOAD는 밟은 칸 수에 선형이어야 한다. 확률 향을 다시 넣지 마라.

[증명 요구] 이 규칙에서 한 바퀴가 반드시 끝난다는 것을 확률이 아니라 산수로 증명해서
문서에 남겨라. 칸이 5개일 때 최대 몇 스텝인지 상한을 숫자로 적어라.

[완료 판정] 각인 조합을 무작위로 수만 번 굴리는 몬테카를로 검사를 만들어

- ① 한 바퀴 상한 초과 0건 ② 한 칸 2회 초과 0건 ③ 스텝 상한 위반 0건
을 출력으로 단언해라. 삭제한 심볼을 누가 되살리면 데이터 검사가 잡게 해라.

AI가 수행한 것 · 검증: 과열 관련 상수 15개 · 함수 4개와 결과 키 전량을 삭제하고

`SLOT_EXEC_CAP = 2` 한 줄로 교체했다. 종료성은 $E = \sum \text{executed}[i]$ 가 매 스텝 최소 1 증가하고 각 칸이 2를 못 넘으므로 $\leq 2n = 10\text{스텝}$ 으로 증명해 문서에 남겼다. 몰빵 방지는 서로 다른 두 축의 사본 감쇠로 처리했다 — 같은 각인을 겹치면 효과 크기 $\times 0.60$ (`DUP_MAG_FALLOFF`), 발동 확률 $\times 0.55$ 가 사본마다 곱해진다. `rune_test.gd` 몬테카를로 25,001 사이클에서 `overload_count=0 exec_cap_violations=0 step_bound_violations=0`, 무작위 최대 스텝이 정확히 10(2n 상한이 여유 없이 딱 맞았다) · 평균 3.86스텝.

4.4 프롬프트 4 — 같은 코드를 다른 모델에게 다시 읽힌다 (CODEX 교차 검증)

상황: 밸런스 상수·전투 판정처럼 되돌리기 비싼 변경을, 그 코드를 쓴 모델의 자기 검토만으로 넘기기 불안한 구간.

(OpenAI Codex CLI에 전달 — 주 에이전트가 만든 diff와 검증 출력을 그대로 넘긴다)

[역할] 너는 이 변경을 처음 보는 리뷰어다. 이 코드를 쓴 것은 다른 모델이고,
그 모델이 남긴 설명은 근거가 아니라 "주장"으로만 취급해라.
"동의한다"가 아니라 "틀린 곳"을 찾는 것이 네 일이다.

[대상] <변경 파일 목록 + diff>

[작성자 주장] <주 에이전트가 남긴 근거 요약과 테스트 출력 원문>

[중점적으로 볼 것]

1. 이 테스트가 정말 이 코드를 물고 있는가. 값이 0이어도 초록불이 나오지 않는가.
2. 배울·상수가 서로 다른 두 경로에서 중복으로 곱해지는 곳은 없는가.
3. 작성자가 "안전하다"고 단정한 근거 중, 실제로는 확인되지 않은 것은 무엇인가.

[출력 형식] 지적마다 ① 파일·줄 ② 무엇이 틀렸는지 ③ 그렇게 본 근거 ④ 확인·재현 방법.
확신이 없으면 "의심"으로 표시해라.

[하지 말 것] 코드를 고치지 마라. 채택 여부는 내가 정하고, 수정은 주 에이전트가 한다.

AI가 수행한 것 · 검증: Codex CLI가 diff와 검증 출력을 **작성자의 설명과 분리해서** 읽고 지적 목록을 냈고, 인간 디렉터가 채택 여부를 판정한 뒤 채택된 지적만 주 에이전트에게 되돌려 수정·재검증하게 했다. 교차 검증이 반복해서 짚은 곳은 이 프로젝트가 자체적으로도 가장 크게 데인 축과 같았다 — **”이 검사가 정말 무언가를 재고 있는가”**. 그래서 그 축은 사람의 주의력에 맡기지 않고 규칙으로 못 박았다. 새 단언에는 **음성 대조**(일부러 틀린 입력으로 빨간불을 확인)를 의무화하고, 각인 ID를 갈아끼울 때는 **부착 수 단언**을 함께 넣게 했다(AGENTS.md 누적 함정 23·26번).

4.5 나머지 지시의 요약

원문 전문 대신 **목표 · 제약 · 완료 판정** 세 줄로 압축했다.

밸런스 프로브 — “숫자를 고치기 전에 근거를 계산하는 도구를 먼저 만들어라”

- **목표** — 화력·몬스터 HP 복리·체류 압박·물량 상한·골드 수지·보스 TTK·마왕 TTK를 절로 나눠 계산하는 헤드리스 `balance_probe` 를 만든다.
- **제약** — 목표 창(관문 30–60초 · 마왕 60–120초 · 15–20일 클리어 · 골드 여유 15–30%)을 프로브가 스스로 판정한다. 프로브가 재는 덱이 실제 게임 덱과 같은지 먼저 확인한다 — 이 조항이, 폐기된 구 각인 ID 때문에 **각인 0개짜리 덱**을 재고 있던 프로브 자체의 결함을 잡아냈다.
- **완료 판정** — 마지막 줄에 판정 8축을 1/0으로 찍고 전부 1일 때만 `pass=1` . 재작성 후 5스테이지 표준 덱 화력이 **119.7 → 394.3**으로 정정됐고, 관문 TTK 실측은 45 / 53 / 56 / 38 / 36 초로 목표 창에 들어왔다.

버그 처리 — “고쳐라”가 아니라 “재현 → 가설 → 확정 → 재발 방지”

- **목표** — 보물상자를 열면 상자 위치 표시가 사라지는 버그(3회 중 3회 재현)를 잡는다.
- **제약** — 바로 고치지 않는다. ①최소 재현 경로를 말로 설명 → ②가설 둘 이상을 코드 근거로 배제·확정 → ③수정 + 같은 부류의 실수가 다른 곳에 있는지 훑기 → ④자동 검사 부착 순서를 지킨다.
- **완료 판정** — “상자 → 성 → 장비 배치·버리기 → 밤 전환”을 잇는 통합 검사 통과와, 원인(상호 작용 사전과 상자 좌표의 **참조 공유**)을 「밝은 함정」에 한 줄로 남기기. 현재는 `--castle-test` / `--event-test`가 승계했다.

자가 검증 체계 — 검사 규격 자체를 AI에게 만들게 하다

- **목표** — 기능을 하나 만들면 그 기능을 무는 headless 검사를 같은 작업에서 함께 만든다. 검사 없는 기능은 “완료”가 아니다.
- **제약** — 커맨드라인 플래그 하나 = 검사 하나이고 끝에 `<이름>_TEST_COMPLETE` 줄을 찍는다. 판정은 문장이 아니라 `키=true` / `키=false`로 낸다. “0개일 때 공허하게 통과하는” 단언을 만들지 않고, 새 단언에는 음성 대조를 함께 돌린다. 재미와 가독성은 검사에서 빼고 캡처 모드로 분리한다.
- **완료 판정** — 러너 `run_all.sh` 종료 코드 0(FAIL 조건은 §5.2의 4조건). 컴파일 1 + 기능 16종 = 요약표 17행 전부 **PASS**, 캡처 73컷 · 고유 지문 72.

인계 문서 — 다음 세션을 위한 산출물

- **목표** — 웨이브를 끝내기 전에 다음 AI가 읽을 인수인계 문서를 코드와 동급의 산출물로 쓴다.
- **제약** — §3.1의 고정 규격(요청·상태·결정·변경 파일·보이는 변화·저장 호환성·검증 출력 원문·밝은 함정·하지 않은 것·다음 시작 지점)을 모두 채우고, `AGENTS.md` 체크포인트와 변경 로그도 함께 갱신한다.
- **완료 판정** — 형식이 고정된 핸드오프 문서 36개. 문서에 기록된 함정이 이후 세션에서 재발한 사례는 없다.

4.6 이미지 생성 프롬프트 — 저작권 안전선을 프롬프트 안에 박다 (실제 원문)

초기 비주얼 방향을 잡기 위해 **OpenAI** 이미지 생성으로 **콘셉트 시트 1장**만 만들었다. 실제로 사용한 프롬프트의 핵심 구문은 다음과 같다.

original top-down 16-bit action RPG
castle, river, bridge, roaming monsters
day-to-night raid
no existing characters or logos

- 저장 위치: `godot-game/art/reference/the-unchosen-visual-direction.png`
- 사용 방식: 런타임 스프라이트로 직접 사용하지 않는다. 색감 · 실루엣 · 낮/밤 대비의 방향 참고로만 썼다.
- `original` 과 `no existing characters or logos` 를 프롬프트에 명시적으로 박은 것이 이 프로젝트 저작권 안전 원칙의 첫 번째 방어선이었다. 생성 결과를 사후에 검열하는 것보다, 생성 조건에 금지선을 넣는 쪽이 확실하다.

5. AI 산출물 검증 체계

구조: AI가 코드를 쓰고 → AI가 그 코드를 무는 테스트를 쓰고 → AI가 테스트를 돌려 결과를 제출하고 → 인간이 화면과 숫자로 최종 승인한다. 되돌리기 비싼 구간은 그 사이에 다른 모델 (Codex CLI)의 리뷰 한 단계를 더 끼웠다.

5.1 명령과 통과 기준

명령	무엇을 보는 가	통과 기준	소요
<code>bash godot- game/scripts/test/run_all.sh</code>	컴파일 1 + 기능 검사 16종	종료 코드 0 · 요약표 17행 전부 PASS	약 135 초
<code>run_all.sh --captures</code>	시각 캡처 15종 73컷 (창을 띄움)	PNG 저장 후 사람 눈 검수	—
<code>balance_probe.gd</code>	화력·HP 복 리·체류 곡 선·물량·경 제·보스/마 왕 TTK (12절)	마지막 줄 <code>pass=1</code> (판정 8축 전부 1)	약 90 초
<code>rune_test.gd</code>	각인 15종 · 사이클 종료 성 몬테카를 로	<code>RUNE_TEST_COMPLETE</code> · 전 플래그 true	—
<code>data_test.gd</code>	데이터 계약 (카드·아이 템·몬스터· 각인 정합 성)	<code>DATA_TEST_COMPLETE</code> · 전 플래그 true	—
<code>rift_probe.gd</code>	균열 배치 · 시드 100개 강건성	<code>failures=0</code> <code>verdict=PASS</code>	—
<code>terrain_probe.gd -- fast</code>	물 비율 · 2x2 블록 통행 불변식	<code>verdict=PASS</code> <code>block_invariant=true</code>	약 40 초

5.2 FAIL 판정 규칙 (기계가 판정한다)

하나라도 걸리면 그 검사는 FAIL이다.

#	조건	뜻
①	Godot 종료 코드 $\neq 0$	테스트가 스스로 실패를 선언
②	기대한 <code>*_TEST_COMPLETE</code> 줄이 없음	도중에 죽었거나 타임아웃
③	출력에 <code>=false</code> 가 있음	검사 플래그 중 하나가 거짓
④	출력에 <code>SCRIPT ERROR</code> / <code>ERROR:</code>	종료 코드가 0이어도 실패로 본다

5.3 실제 검증 로그 (최종 밸런스 웨이브 Y8)

```
BALANCE_PROBE_COMPLETE pass=1 rune_steps=1 hp_index=1 dwell_curve=1
                        dwell_pressure=1 volume=1 gold=1 boss_ttk=1
demon_ttk=1
STAGE curve=true reward_decay=true eff4=0.772 eff8=0.611 eff12=0.500
night_cap=62
CYCLE exec_cap=true overload_hits=0 max_steps=10 max_exec=2 debt=2.42
STRESS fps=145 enemies=103 surge_peak=59 surge_cap=78
BOSS stage_reload=1.485 expected_reload=1.485 demon_reload=0.873
DATA rune_exec_cap=true slot_exec_cap=2 step_cap=12 runes=15
RUNE no_overload=true overload_count=0 guard_count=0 cycles=25001
```

- `step_cap=12` 는 엔진 쪽 안전 가드다. 규칙 상한(5칸 × 2회 = **10스텝**)보다 일부러 여유를 둔 값이며, 실제 최대 스텝은 위 `max_steps=10` 처럼 규칙 상한에서 멈춘다.
- `run_all.sh` **17행 전부 PASS · 2회 연속 재현**
- 몬스터 **103마리** 동시 유지 상태에서 **145 FPS** (성능 회귀 검사)
- 각인 사이클 **25,001회** 몬테카를로에서 상한 초과 0건

5.4 작업 종료 전 체크리스트 (`AGENTS.MD` §13에 규정)

AI 세션은 작업을 끝내기 전에 이 세 가지를 반드시 통과해야 한다.

1. `run_all.sh` 전 검사 PASS (**17/17**)
2. 밸런스를 건드렸으면 `balance_probe` 재실행 → `pass=1`
3. 인간 디렉터의 최종 승인

인간의 최종 승인 없이는 어떤 웨이브도 “완료”가 되지 않는다. 자동 검사는 “AI가 인간의 시간을 낭비하지 않을 자격”을 증명하는 절차였다.

6. 저작권 · 에셋의 AI 관련 처리

6.1 원칙

원칙	실행
기존 작품 복제 금지	기존 게임의 인물·몬스터·UI·로고·음악·고유 장소를 복제하지 않는다. 이미지 생성 프롬프트에 <code>no existing characters or logos</code> 를 명시한다
런타임 그래픽은 오리지널	Godot Draw API로 그린 오리지널 픽셀 도형을 쓰고, 런타임 PNG는 코드로 굽는다 (<code>art/v2/build_assets.y.gd</code> 가 15장 생성한다)
AI 이미지 생성물은 런타임 미사용	콘셉트 시트 1장뿐이며 방향 참고로만 쓰고, 최종 빌드에는 넣지 않는다
외부 게임 에셋 0건	상용·유료 게임 에셋은 쓰지 않는다
효과음은 런타임 합성	<code>AudioStreamWAV</code> 샘플을 코드에서 합성한다
배경 음악은 생성형 AI 산출물	Gemini로 원곡을 생성하고 Suno로 리믹스한다. 기존 곡을 샘플링하지 않는다

6.2 사용 에셋과 라이선스

에셋	제작	라이선스	용도
Ninja Adventure Asset Pack	Pixel-boy & AAA	CC0	런타임 스프라이트
Kenney Particle Pack	Kenney	CC0	이펙트
갈무리11 (Galmuri11)	quiple	OFL 1.1	UI 폰트, 웹 빌드에 임베드
coi-serviceworker (v0.1.7)	Guido Zuidhof	MIT	GitHub Pages COOP/COEP 헤더 주입
game-icons 113 장	작가 9명	CC BY 3.0	저장소에만 포함(전문 <code>godot-game/art/external/LICENSES.md</code>), 게임 빌드에서는 제외
BGM	AI 생성 (Gemini 원곡 · Suno 리믹스)	AI 생성물	배경 음악

참고 — coi-serviceworker는 게임 런타임 로직이 아니라 배포 인프라용이다. 웹 빌드가 스레드를 쓰기 때문에 필요한 COOP/COEP 헤더를, 커스텀 헤더를 지정할 수 없는 GitHub Pages에서 서비스 워커로 대신 주입한다(Vercel은 `vercel.json`으로 직접 지정). MIT 표기 의무는 `godot-game/web_extras/coi-serviceworker.LICENSE.txt`에 전문을 동봉해 이행했다.

참고 — CC BY 3.0 아이콘 팩은 저장소에만 남기고 게임 빌드에서는 제외했다. 빌드 산출물이 의존하는 파일이 0개라 게임 크레딧의 표기 의무는 발생하지 않으며, 작가 9명의 목록(Lorc 68 · Delapouite 31 · Sbed 5 · Skoll 3 · Willdabeast 2 · Felbrigg 1 · DarkZaitzev 1 · Cathelineau 1 · Carl Olsen 1)은 저장소 `LICENSES.md` §6에 그대로 둔다.

6.3 AI 생성물의 저작권 관점 정리

- **코드**: 주 개발 에이전트(Claude Code)가 작성했고, 위험 구간은 OpenAI Codex CLI의 교차 리뷰를 거쳤다. 요구사항·구조·판단 기준은 인간 디렉터가 제시했고 모든 결과물은 인간이 검토·승인했다. 외부 코드의 무단 복제는 없다.
- **이미지**: AI 생성 이미지는 **최종 빌드에 포함되지 않는다**(참고 1장, `art/reference/`).
- **음원**: 배경 음악은 Google Gemini로 원곡을 생성하고 Suno로 리믹스한 **생성형 AI 산출물**이며, 기존 곡을 샘플링하거나 특정 곡을 재현하지 않았다. 효과음은 코드에서 합성한 오리지널이다.
- **에셋 매핑 전체 표**: `godot-game/art/v2/ASSET_MAP.md`

7. 정리

8일 만에 이 볼륨이 나온 이유는 코드 생성 속도 때문이 아니라, *****AI가 자기 작업을 스스로 검증하고, 다음 AI에게 정확히 인계하는 구조*****를 초반에 세워 뒀기 때문이다. 인간의 검토 시간이 병목이 되지 않았다. 되돌리기 비싼 구간에만 **다른 모델(Codex CLI)의 리뷰**를 한 겹 더 얹어, “한 모델이 자기 실수를 자기 눈으로 찾아야 하는” 상황을 줄였다.

7.1 제목의 변천 — 반복 개발의 증거

시점	제목	그때의 게임
2026-08-03	THE UNCHOSEN: NIGHTFALL	낮 탐험 / 밤 습격 · 버린 스킬이 마왕에게 가는 구조
2026-08-04	딜사이클 용사	동시 발동 폐기 → 왼쪽에서 오른쪽으로 도는 레일 전투 로 전환
2026-08-10	극딜 용사	5칸 · 각인 15종 · 속성 콤보 · 5스테이지 관문 구조 확정

제목이 두 번 바뀐 이유는 게임의 **중심 규칙이 두 번 같렸기** 때문이다. AI와의 개발이 “한 번에 완성”이 아니라 **디렉팅 → 구현 → 실제 플레이 → 재설계**의 반복이었다는 증거로 그대로 남겨 둔다. 개발 라운드도 같은 흔적을 남겼다 — W(v2) → V(v3) → U(UI) → X(1차 피드백) → Y(2차 피드백 25건), 총 36개 웨이브다.

7.2 인간 디렉팅이 결정적이었던 지점

AI가 스스로 도달하지 못했고, **인간의 한 문장이 프로젝트 전체를 바꾼** 지점은 넷이다.

1. **정체성** — “고르지 않은 선택이 마왕에게 돌아온다.” AI는 이 축이 없을 때 장르 자체를(벽돌깨기로) 바꿔 버렸다.
2. **중심 규칙 교체** — “동시 발동을 폐기하고 레일로 바꿔라.” 플레이어가 자기 행동을 볼 수 있게 된 순간이다.
3. **복잡도 제거** — “과열을 없애라”, “블록을 다 지워라”, “텍스트를 반으로.” AI는 기능을 더하는 쪽으로는 잘 가지만, **덜어내는 판단은 인간이 내렸다.**
4. **재미의 판정** — “검사는 통과했는데 어렵지가 않다 / 안 읽힌다.” headless 검사는 무너짐만 잡는다. 재미·가독성·난이도는 실제 창에서만 판정됐다.

7.3 이 프로젝트가 제안하는 AI 활용 방법론 3줄

① 세션은 끝긴다 — 인계 문서를 산출물로 취급한다. ② AI가 만든 것은 AI가 만든 검사로 잡
되, 그 검사가 무엇을 재는지 인간이 의심한다. ③ 감각적 요구(“가린다”, “복잡하다”)를 숫자 계
약으로 번역하면 회귀가 자동으로 잡힌다.

재작업을 줄인 것은 모델의 성능이 아니라 프롬프트에 미리 넣어 둔 판정 기준이었다. 금지선(“장르를 바꾸지 마라”), 상한과 가드(“한 칸은 한 바퀴에 두 번까지”), 검증 방법(“확률이 아니라 산수로 증명해라”), 다음 세션을 위한 기록, 그리고 다른 모델의 재검토까지 지시문 안에 함께 넣어 두었다.